

System Architecture Document

Student Bus Tracking PWA

Version: 1.0
Project: Student Bus Tracking System
Frameworks: React (Frontend), Fastify (Backend)

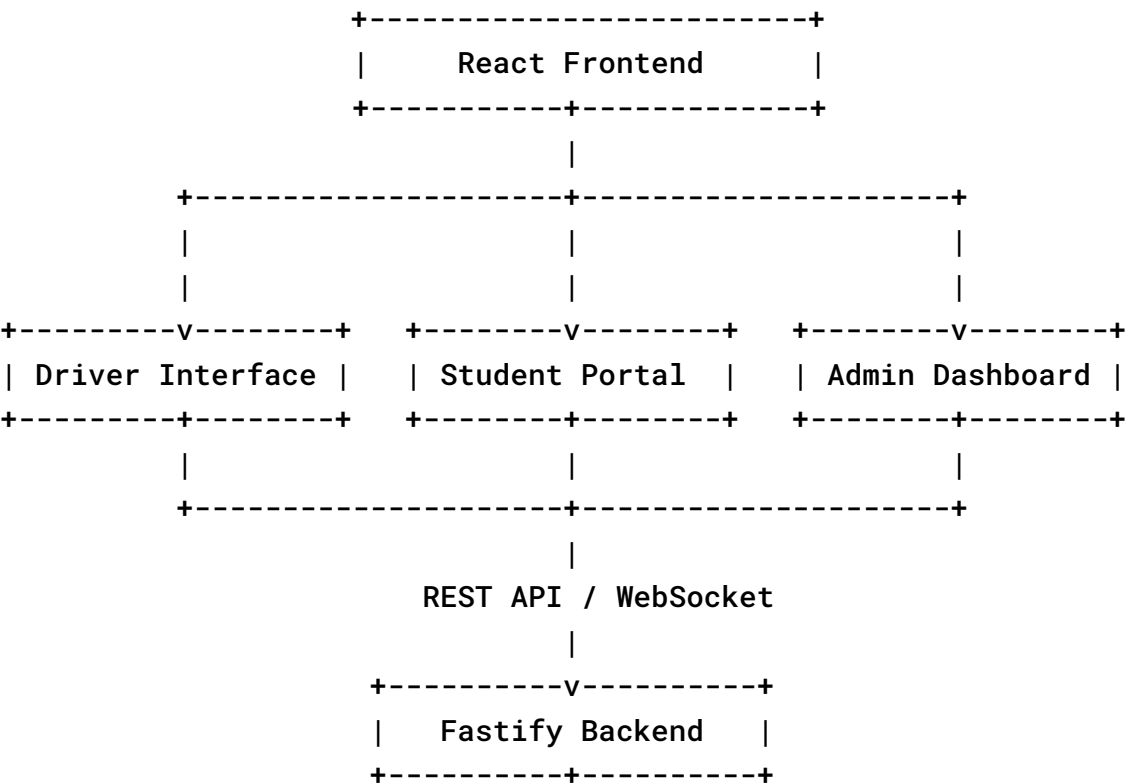
1. Introduction

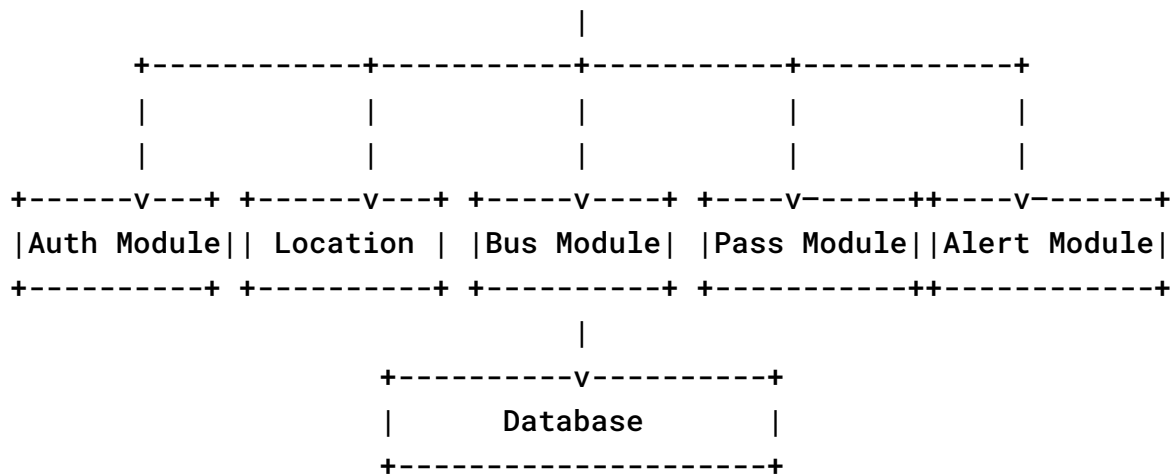
This document presents the high-level system architecture for the Student Bus Tracking Progressive Web Application (PWA). The system is designed to provide real-time bus location tracking, automated alerts, and comprehensive administrative management for student transportation. This architecture follows standard software design principles to ensure scalability, maintainability, and reliability.

2. System Overview

The system utilizes a hybrid architectural pattern combining Client-Server architecture for transactional operations (pass renewal, route management) and the Publish-Subscribe (Pub-Sub) pattern for real-time location streaming.

2.1 High-Level Component View



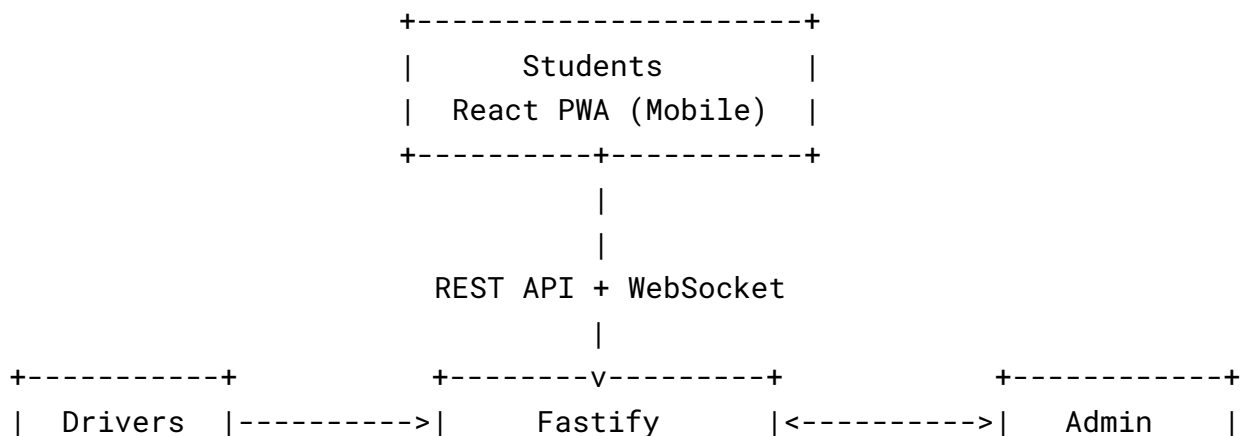


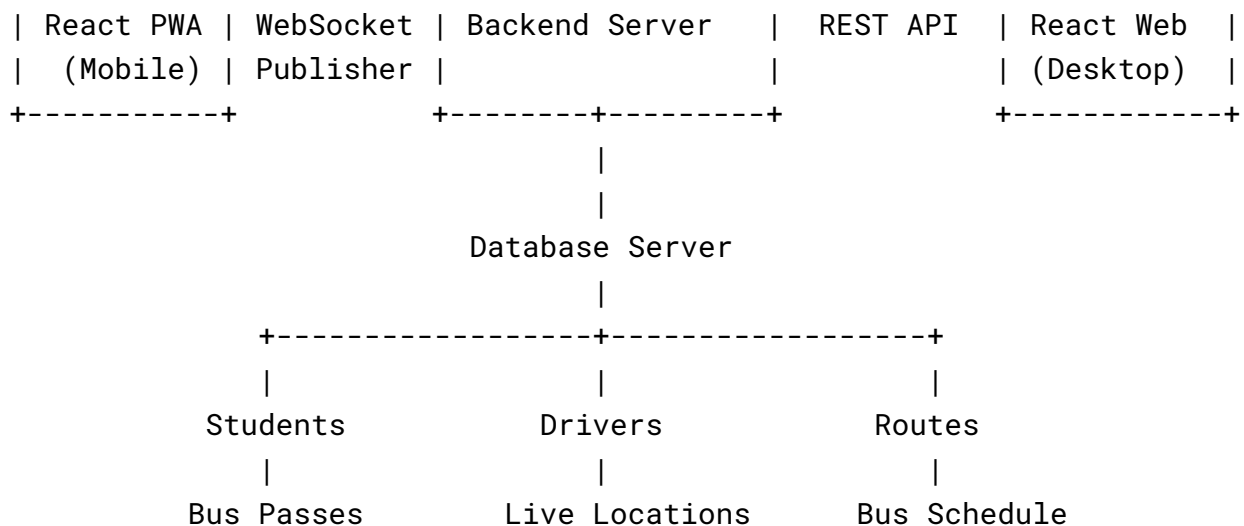
- **Client Layer:** React-based PWA providing tailored interfaces for three distinct user roles (Driver, Admin, Student).
- **Application Layer (Backend):** Fastify server implementing RESTful APIs for CRUD operations and WebSocket services for real-time telemetry.
- **Data Layer:** Centralized database for storing persistent records (student passes, bus routes, schedule data).

3. User Role & Functional Architecture

Role	Interface	Key Responsibilities
Driver	Mobile PWA	Start/Stop trip (Pub-Sub initiation), SOS/Breakdown triggers.
Admin	Desktop PWA	Route management, student pass renewal, bus availability, schedule updates.
Student	Mobile PWA	Real-time map view, route visualization, bus pass access, 5km proximity alerts.

4. Architectural Patterns





4.1 Client-Server Architecture

Used for standard requests where the client initiates a request (e.g., "Renew my pass," "Update bus timing") and the server processes the logic, interacts with the database, and returns a response.

4.2 Publish-Subscribe (Pub-Sub) Pattern

Critical for the real-time location feature.

Critical for the real-time location feature.

- **Publisher:** The Driver's mobile app publishes location coordinates and trip status events to the Fastify WebSocket server upon starting the trip.
- **Subscriber:** The Student's mobile app subscribes to specific bus channels. The server broadcasts the Driver's location updates to all connected Student clients currently observing that specific bus/route.¹

5. System Workflow & Data Flow

5.1 Real-Time Tracking Flow

1. **Start Trip:** Driver triggers the "Start" event. The WebSocket connection is established to the server.¹
2. **Broadcasting:** The Driver's app sends periodic GPS coordinates.
3. **Distribution:** The server receives the stream and broadcasts the updated coordinates to all subscribed student clients.
4. **Vicinity Logic:** The Student app performs proximity calculation (Client-side) or the server filters events (Server-side) to trigger a push notification when the bus enters the 5km vicinity.

5.2 Admin Management Flow

- Admin creates or modifies route/timing data via the Desktop PWA.
- The system updates the central database and notifies the affected driver and student clients via the API layer.¹

6. Component & Connector (C&C) View

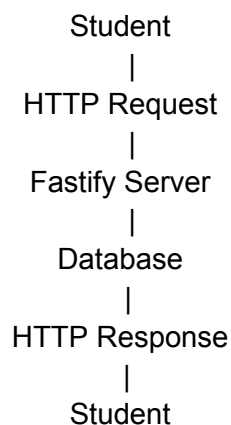
The system is divided into functional components governed by clearly defined connectors:

Components:

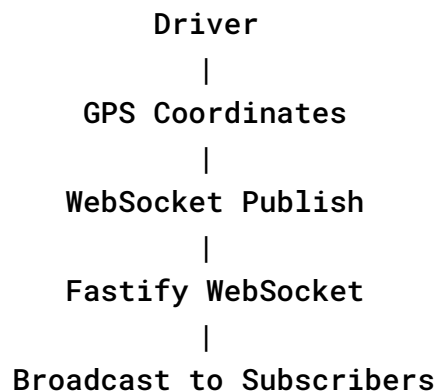
- **Components:**
 - *Auth Module*: Handles role-based access control (RBAC).⁵
 - *Bus/Route Service*: Manages CRUD operations for static scheduling.
 - *WebSocket Service*: Handles real-time telemetry streams.
 - *Notification Service*: Handles vicinity alerts and system updates.

Connectors:

- *HTTP/REST API*: Connects Client to Server for non-real-time data.⁴



- *WebSockets*: Connects Driver/Student apps for real-time location data.³



/ | \
Student A Student B Student C

7. Non-Functional Requirements

To ensure the system meets its operational goals, the following attributes are prioritized:

To ensure the system meets its operational goals, the following attributes are prioritized:¹⁵

- **Scalability:** The architecture supports horizontal scaling by leveraging the independent nature of Fastify's event loop and containerized deployment.¹³
- **Fault Isolation:** By separating the real-time WebSocket service from the management API, a failure in one does not cascade to the other.¹³
- **Usability:** Mobile-first design for students and drivers ensures accessibility for users on the move.⁵

8. Summary of Architectural Decisions

- **React + Fastify:** Chosen for high performance in I/O bound operations (WebSockets) and developer productivity.
- **Database per Service:** If the system expands, each microservice (User, Bus, Payment) should ideally maintain its own data store to ensure loose coupling.³

This architecture provides a blueprint for implementation, bridging the requirements of the student transportation system with robust software engineering practices.